

mCOLD — สิ่งที่ต้อง Server ต้องทำ (MQTT)

เวอร์ชัน 0.2 (ฉบับร่าง) · 2026-10-04 · firmware 0.7.0-dev

เอกสารนี้สรุปสำหรับทีม server: กล้อง mCOLD ส่งอะไรขึ้นมา, server ต้องตอบอะไรกลับ, และเรื่องที่ต้องตัดสินใจร่วมกัน ฝั่งกล้องทำเสร็จและเชื่อมต่อ broker ของทีมได้แล้ว **สิ่งที่ยังขาดคือ ACK จาก server** รายละเอียดเต็มอยู่ใน `firmware/PROTOCOL.md` ข้อ 6

1. สิ่งที่ต้องทำ เรียงตามความสำคัญ

#	เรื่อง	ถ้ายังไม่ทำ
1	ส่ง ACK หลังเก็บข้อมูลลงฐานข้อมูลแล้ว (ข้อ 4)	กล้องไม่ลบข้อมูลเลย ส่งชุดเดิมซ้ำ และเว้นระยะการส่งนานขึ้นถึง 4 ชั่วโมง ข้อมูลใหม่ขึ้นซ้ำมาก
2	ACK ให้เร็ว ภายใน 1-2 วินาที	ทุกวินาทีที่รอคือวิทยุ Wi-Fi เปิดค้างในทุกกล้อง กินแบตเตอรี่โดยตรง
3	เก็บแบบไม่ซ้ำ โดยใช้ (sn, trip, seq) เป็น key	กล้องส่งซ้ำทุกครั้งที่ไม่ได้ ACK ข้อมูลจะซ้ำในฐานข้อมูล
4	เปิด TLS (8883) และ login แยกต่อเครื่อง	ตอนนี้ข้อมูลและรหัสผ่านวิ่งแบบไม่เข้ารหัส และใครมี login เดียวกันก็ปลอม ACK ได้ ทำให้กล้องลบข้อมูลที่ยังไม่ถึง server
5	ตอบเรื่อง OTA (ข้อ 7)	ยังอัปเดต firmware ทางอากาศไม่ได้

2. การเชื่อมต่อ

- client id = รหัสเครื่อง (sn) ตามฉลาก เช่น mCDV1-L0169-1069-001 (ตัวพิมพ์เล็ก-ใหญ่มีผล) กล้องที่ยังไม่ได้ตั้ง SN จากโรงงานจะใช้ชื่อชั่วคราว MCOLD-xxxx
- broker และ login ตั้งในเครื่องแต่ละกล้อง ไม่อยู่ใน firmware หรือ git
- ทุก topic อยู่ใต้ mcold/<sn>/

Topic	ทิศทาง	QoS	Retained	เนื้อหา
mcold/<sn>/rec	กล้อง → server	1	ไม่	ชุด record (ข้อ 3)
mcold/<sn>/ack	server → กล้อง	1	ไม่	{"trip":T,"upto":S} (ข้อ 4)
mcold/<sn>/status	กล้อง → server	0	ใช่	สถานะล่าสุด (JSON เดียวกับ GET_STATUS)
mcold/<sn>/online	กล้อง → server	1	ใช่	"1" ตอนเชื่อมต่อ, "0" ตอนจบรอบหรือหลุด (last will)

รับข้อมูลทุกกล้องได้ด้วยการ subscribe `mcold+/rec`

3. ข้อมูลที่กล้องส่งขึ้นมา

```
{ "sn": "mCDV1-L0169-1069-001", "trip": 8, "schema": 1, "from": 0, "to": 15, "records": [{"seq": 0, "type": 1, "data": "<base64>"}, ... ] }
```

- ชุดละไม่เกิน 16 record ของ trip เดียว เรียงตาม seq
- data คือ record แบบ binary (little-endian) เข้ารหัส base64
- type: 1 = เริ่ม trip, 2 = sample, 3 = event, 4 = จบ trip
- ทุก record ขึ้นต้นด้วย stamp 11 byte

Offset	ชนิด	ความหมาย
0	u32	เวลา UTC (วินาที) — ใช้ไม่ได้ถ้า quality = 0
4	u8	quality ของเวลา: 0 ไม่มี, 1 RTC, 2 GNSS, 3 แอป, 4 NTP
5	u16	boot (นับเฉพาะการรีเซ็ตจริง)
7	u32	tick (ms) — boot + tick ใช้เรียงลำดับ record ของกล่องเดียวกัน

Sample (type 2) — 1 record ทุก 5 นาทีระหว่าง trip

Offset	ชนิด	ความหมาย
11	u8	สถานะการวัดอุณหภูมิ (0 = ปกติ)
12	u16	ค่าดิบ MAX6675 (หน่วย 0.25 °C); 0xFFFF = ไม่มี
14	i16	อุณหภูมิหลังสอบเทียบ (0.01 °C); -32768 = ไม่มี
16	u8	ประตู: 0 ไม่ทราบ, 1 ปิด, 2 เปิด (ฟังก์ชันประตูปิดอยู่)
17	u16	จำนวนครั้งที่เปิดประตูใน trip นี้
19	u16	จำนวนครั้งที่ขยับ ตั้งแต่ sample ก่อนหน้า
21	u8	GNSS: bit0 fix รอบล่าสุด, bit1 เคย fix
22 / 26	i32 / i32	latitude / longitude (1e-7 องศา)
30	u16	อายุของ fix (วินาที); 0xFFFF = ไม่มี
32 / 33	u8 / u8	ดาวเทียมที่ใช้ / HDOP×10 (0xFF = ไม่ทราบ)
34	u16	แรงดันแบตเตอรี่ (mV); 0 = ไม่ทราบ
36	u8	แบตเตอรี่ %; 0xFF = ไม่ทราบ
37	i16	กระแสแบตเตอรี่ (mA, + = เข้าแบตเตอรี่); -32768 = ไม่ทราบ
39	u8	สถานะการชาร์จ
40	u16	alarm ที่ active (bit ต่อ alarm)
42	u16	อุปกรณ์ที่ทำงานผิดปกติ (bit ต่ออุปกรณ์)

เริ่ม trip (type 1) — มีช่วง alarm, รอบการบันทึก, ค่าสอบเทียบ, เวอร์ชัน firmware และ SN (offset 58, ยาว 24 byte ใน header format 2 — format 1 เดิมยาว 12 byte)

Event (type 3) — byte 11 คือรหัส event: 1 รีเซ็ตแล้วทำ trip ต่อ, 4 ขยับ, 5 probe เสีย, 6 probe กลับมาปกติ, 7 alarm เกิด, 8 alarm หาย, 9 รับทราบ alarm, 10 ตั้งเวลา, 11 ข้อมูลถูกลบเพราะพื้นที่เต็ม, 12 ปิดเครื่องเพราะแบตเตอรี่หมด (2, 3 เป็นเรื่องประตูซึ่งปิดอยู่) รายละเอียดทุกฟิลด์อยู่ที่ `firmware/src/record.h`

4. ACK — สิ่งที่ server ต้องตอบ

- เก็บ record ของชุดนั้นลงฐานข้อมูลให้เรียบร้อย **ใช้ (sn, trip, seq) เป็น key record** ที่มาซ้ำต้องเก็บครั้งเดียว
- จากนั้น publish ไปที่ `mcoId/<sn>/ack`

```
{"trip":8,"upto":15}
```

แปลว่า **record ของ trip 8 ตั้งแต่ seq 0 ถึง 15 เก็บครบแล้ว** ต้องต่อเนื่อง ห้ามตอบ upto ถ้ายังมี seq ที่น้อยกว่านั้นขาดอยู่

- **PUBACK ของ MQTT ไม่นับเป็น ACK** เพราะบอกแค่ว่า broker ได้รับ ไม่ใช่ว่า server เก็บแล้ว

- กล้องตรวจ ACK ทุกครั้ง: ถ้าเป็น trip ที่ไม่มี หรือ upto เกินข้อมูลที่มี จะไม่รับ
- ACK ช้าหรือมาช้าไม่เสียหาย เพราะค่านี้เดินหน้าอย่างเดียว

5. จังหวะการส่งเมื่อกล้องใช้แบต

กล้องหลักระหว่างรอบเพื่อประหยัดแบต และ Wi-Fi ปิดอยู่ จะเชื่อมต่อเป็น **รอบสั้นๆ** ทุก 5 นาที เมื่อมีข้อมูลรอส่ง:

1. เปิด Wi-Fi → เชื่อม broker → ส่ง status → ส่งชุด record
2. **รอ ACK ชุดละไม่เกิน 10 วินาที** ได้ ACK แล้วส่งชุดถัดไป
3. จบรอบ: ส่ง "0" ที่ online แล้วตัดการเชื่อมต่อ

สถานการณ์	กล้องทำอะไร
server ACK ทันเวลา	รอบถัดไปตามปกติใน 5 นาที
ไม่ได้ ACK ในรอบนั้น	เว้นระยะรอบถัดไปเป็น 10, 20, 40 ... นาที สูงสุด 4 ชั่วโมง
ACK มาหลังจบรอบแล้ว	ACK นั้นหายไป (กล้องใช้ clean session) รอบหน้ากล้องส่งชุดเดิมซ้ำ server แะ ACK อีกครั้ง

สรุปสำหรับ server:

- ACK ภายใน 1-2 วินาที ทำให้แต่ละรอบใช้เวลาราว 5 วินาที
- ดูว่ากล้องยังติดต่อกได้จากเวลาของ status ล่าสุด ไม่ใช่จาก online เพราะกล้องที่ใช้แบตจะเป็น "0" เกือบตลอดเวลาและถือว่าปกติ
- ตอนเสียบสาย USB กล้องเชื่อมต่อค้างไว้ตลอด ใช้ทดสอบฝั่ง server ได้สะดวก

6. ความปลอดภัย — ขอให้เปิดก่อนใช้งานจริง

- **TLS ที่พอร์ต 8883** พร้อม certificate ของ server ที่กล้องตรวจสอบได้
- **login แยกต่อเครื่อง** (หรือ client certificate)
- **ACL:** แต่ละกล้อง publish ได้เฉพาะ mcold/<sn>/rec , status , online ของตัวเอง และ subscribe ได้เฉพาะ mcold/<sn>/ack ของตัวเอง ส่วน ack ให้ publish ได้เฉพาะ server

เหตุผล: ACK คือคำสั่งให้กล้อง "ลบข้อมูลได้" ถ้าใครปลอม ACK ได้ ข้อมูลที่ยังไม่ถึง server จะหายถาวร

7. คำถามที่ต้องการคำตอบ

1. ยอมรับรูปแบบ topic และ ACK ตามนี้ไหม หรืออยากปรับตรงไหน
2. เปิด TLS 8883 และ login แยกเครื่องได้เมื่อไหร่
3. OTA: จะให้กล้องดาวน์โหลด firmware จาก file server (HTTPS) หรือส่งผ่าน MQTT (ไม่ว่าแบบไหน firmware จะถูกเซ็นและกล้องตรวจลายเซ็นก่อนติดตั้ง)
4. ใครเป็นผู้ติดต่อฝั่ง server สำหรับทดสอบร่วมกัน

เอกสารอ้างอิง

- firmware/PROTOCOL.md ข้อ 6 — สเปกเต็มของ MQTT
- firmware/src/record.h — โครงสร้าง record ทุกชนิด